8. Instantiate (crear objetos en tiempo de ejecución)

Instantiate() es una de las funciones más importantes de Unity: permite **crear objetos durante el juego**.
En este proyecto se utiliza para generar balas, explosiones, rocas y otros elementos dinámicos.

8.1. ¿Qué es Instantiate?

Instantiate() toma un **prefab** o un objeto ya existente y crea una **copia** en la escena.
Esta copia mantiene componentes, comportamientos y propiedades del original.

La forma más habitual de uso es:

```
Instantiate(prefab, posicion, rotacion);
```

8.2. Ejemplos en los scripts del proyecto

8.2.1. Creación de balas (script Jugador)

```
void CrearBala(Vector3 desplazamiento, float angulo)
{
    Instantiate(prefabBala, transform.position + desplazamiento,
Quaternion.Euler(0, 0, angulo));
}
```

Aquí se instancia una bala en una posición desplazada respecto al jugador con una rotación concreta.

8.2.2. Enemigo disparando balas (script Enemigo)

```
void CrearBala()
{
    Instantiate(prefabBalaE, transform.position + Vector3.down,
Quaternion.Euler(0, 0, -90));
}
```

Esta bala siempre se instancia debajo del enemigo y con una rotación hacia abajo.

8.2.3. Crear explosiones al morir un enemigo

Instantiate(prefabExplosion, transform.position, Quaternion.identity);

Esto genera un efecto visual en el lugar donde el enemigo ha sido destruido.

8.2.4. Spawn de rocas aleatorias (script GameManager)

```
Vector3 posicion = new Vector3(Random.Range(-9f, 9f),
Random.Range(-5f, 5f), 0);

Instantiate(prefabRoca, posicion, Quaternion.Euler(0, 0,
Random.Range(0, 360)));
```

Cada roca aparece:

- En una posición aleatoria
- Con una rotación aleatoria

8.3. Parámetros de Instantiate

Forma completa:

Instantiate(original, position, rotation, parent);

- original: el prefab u objeto original
- position: Vector3 de destino
- rotation: Quaternion
- parent: tipo Transform, opcional, para crearlo como hijo del transform especificado

Forma básica:

Instantiate(original);

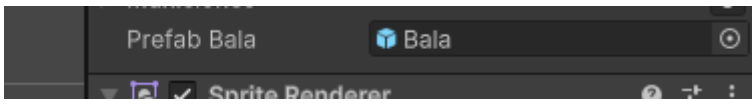
Clone sin cambiar posición ni rotación.

8.4. Usar Instantiate con prefabs

El patrón habitual en Unity es:

```
[SerializeField] GameObject prefab;
```

Asignar el prefab en el Inspector…



…y luego instanciarlo desde el código.

Esto garantiza que:

- El prefab existe
- Tiene todos los componentes configurados
- No hay errores de referencias nulas

8.5. Controlar la rotación (Quaternion)

Rotar mediante grados Euler:

```
Quaternion.Euler(0, 0, angulo);
```

El script Jugador lo usa para disparar hacia distintas direcciones.

8.6. Instanciar con desplazamiento local

Si queremos que el objeto aparezca relativo a otro:

transform.position + desplazamiento

Ejemplo del disparo triple:

```
CrearBala(new Vector3(0.4f, 0, 0), 60);
```

Esto permite crear patrones de disparo fácilmente.

8.7. Buenas prácticas

- Mantener prefabs bien organizados
- No instanciar en Update si no es necesario
- Destruir los objetos que ya no se usen (ver ficha Destroy)
- Evitar instanciar demasiados objetos simultáneamente
- En shooters o juegos con muchos proyectiles: usar **Object Pooling**

8.8. Errores comunes

- Olvidar asignar el prefab en el Inspector → NullReferenceException
- Instanciar cientos de objetos por segundo → bajada de FPS
- Instanciar dentro de OnCollisionEnter sin control → cascada de objetos inesperados